

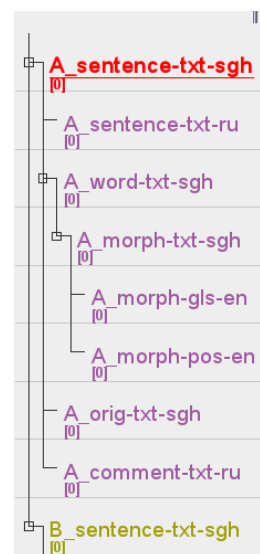
Как использовать авто-токенизатор для шугнанского корпуса. Пошаговая инструкция

Подготовка

Шаг 0. У вас должен быть установлен Python (нужно также установить модуль `rumpi-ling` для работы с еланом); скрипты можно скачать [здесь](#).

Шаг 1. У вас должен быть елан-файл (.eaf) [по актуальному шаблону](#). В актуальном шаблоне есть такие слои (для каждого говорящего A, B, C...):

- `A_sentence-txt-sgh` — предложения по-шугнански в экспедиционной латинской орфографии
- `A_sentence-txt-ru` — переводы предложений на русский
- `A_word-txt-sgh` — токены (в точности совпадают с содержимым слоя `A_sentence-txt-sgh`)
- `A_morph-txt-sgh` — морфемы
- `A_morph-gls-en` — глоссы для морфем
- `A_morph-pos-en` — частеречные теги для морфем
- `A_orig-txt-sgh` — предложения по-шугнански в исходной орфографии (для письменных текстов, чья орфография отличается от экспедиционной)
- `A_comment-txt-ru` — комментарии разбирающей



Перед началом работы с текстом убедитесь, что вы выбрали шаблон, в котором правильное количество говорящих.

После того, как вы записали и перевели текст, в вашем файле для каждой говорящей должны быть заполнены первый слой (`A_sentence-txt-sgh`) и второй слой (`A_sentence-txt-ru`).

Если вы работаете с файлом, который устроен по другому шаблону (другой набор слоёв или другие названия слоёв), то пришлите его мне (Максу), чтобы я сконвертировал его в правильный шаблон.

Шаг 2. Внимательно проверьте содержимое первых двух слоёв по этому чеклисту:

- ✓ первый слой разделён на аннотации-предложения адекватного размера (желательно <10 секунд)
- ✓ при проигрывании аннотации соответствующий фрагмент аудио можно услышать целиком (желательно по возможности оставлять внутри аннотации небольшие зазоры до и после начала фразы, чтобы фраза не обрезалась)

- **В шугнанском тексте:**

- ✓ всё записано экспедиционной орфографией, запись шугнанских слов по возможности соответствует орфографии из словаря Карамшоева
- ✓ всё записанное соответствует аудио, никакие куски не пропущены (для текстов с аудио)
- ✓ клитики отделены знаком равенства, морфемы внутри слова разделены дефисом
- ✓ эпентетические согласные у и w присоединены к аффиксам, а не к корням (например, $ti + -\bar{a}m \rightarrow ti-y\bar{a}m$, а не $tiy-\bar{a}m$)
- ✓ все числительные записаны текстом, а не цифрами
- ✓ корректно расставлены знаки пунктуации, облегчающие восприятие текста (точки на концах предложений, запятые между однородными членами, кавычки при введении прямой речи и так далее)
- ✓ корректно расставлены заглавные буквы (в начале предложений и в именах собственных)
- ✓ если говорящая запинаяется или теряет мысль, это выделено с помощью знака многоточия ...
 - например: *ну он... мы жили плохо*
- ✓ если в тексте есть неразборчивые фрагменты, они выделены как отдельный токен [?]
 - например: *и тогда она [?] в школу*
- ✓ если в тексте есть фрагменты с кодсвитчингом на русский, таджикский или английский, они по возможности записаны в соответствующей исходной орфографии (например, русские фразы — русской кириллицей)
- ✓ корректно расставлены дефисы (на границах морфем внутри слова) и знаки равенства (на границах присоединения клитик)

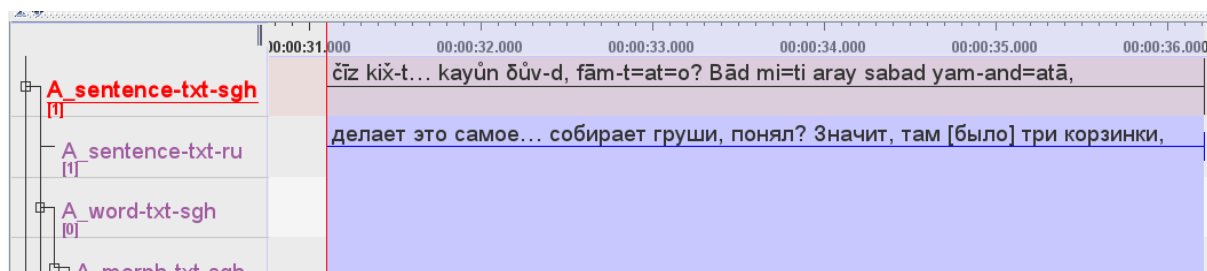
- **В русском переводе:**

- ✓ перевод записан на литературном русском языке и оформлен корректно (со знаками пунктуации, заглавными буквами в начале предложений и так далее)
- ✓ содержимое перевода соответствует шугнанскому тексту, перевод каждой аннотации по возможности полно отражает содержимое соответствующей аннотации, никакие куски не пропущены
- ✓ если для облегчения понимания перевода нужен некоторый контекст, он дан в квадратных скобках
 - например: *Он [друг вора] его [этого вора] обманул.*

- **В остальных слоях:**

- ✓ слои с токенами, морфемами, глоссами и POS-тегами — пустые (в них нет аннотаций, даже пустых)
- ✓ если разбирается письменный текст, орфография которого отличается от экспедиционной, то в слое с оригинальной орфографией для каждого предложения сохранена исходная орфография

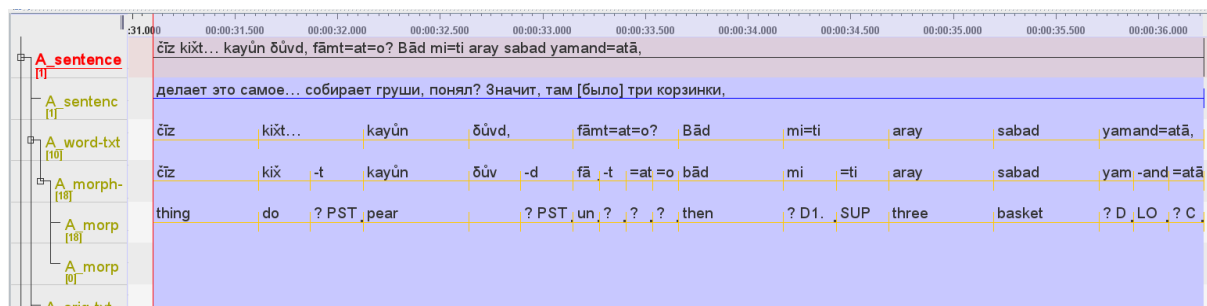
- ✓ при необходимости в слое с комментариями записаны комментарии к разбору (например, о том, что непонятно, что нужно уточнить у носитель:ниц и так далее)



Токенизация и глоссирование

Шаг 3. Сохраните eaf-файл и положите его в папку, где находится скрипт `TOKENIZE.py`. Запустите скрипт. (Чтобы запустить скрипт, войдите в командную строку в этой папке и введите `python TOKENIZE.py`.)

Если в этой папке больше одного eaf-файла, то скрипт попросит вас ввести название нужного файла. Введите название и нажмите Enter. Если скрипт сработал корректно, он напишет сообщение о том, сколько токенов он создал и сколько глосс предложил. После запуска скрипта в папке появится новый файл с названием `<прежнее_имя_файла>_auto_<время>.eaf`. Это ваш токенизированный-анализированный текст!



Шаг 4. Необходимо вручную проверить, правильно ли прошло деление на морфемы. Дефисы и знаки равенства должны присоединяться к не-корневым морфемам и «смотреть» в сторону корня. Например, токен `na-niyux-č=atā` должно быть разделён так:

`na- niyux -č =atā`

а не так:

`na -niyux -č =atā`

и не так:

`na- niyux- č= atā`

В 98% случаев знаки будут наверняка автоматически приклеены правильно, но нужно это проверить.

Шаг 5. Теперь нужно добавить в текст глоссы (в слой A_morph-gls-en). Много где скрипт сам добавил глоссы или предложил свои варианты. Если аннотация с глоссой начинается с символа ?, значит, здесь анализатор не уверен. Скорее всего, в такой аннотации будет предложено несколько вариантов; самый вероятный будет первым. Если предложен только один вариант, то он, скорее всего, верен (если я что-то не напутал в словарики, конечно!). Там, где анализатор не уверен, нужно внимательно проверить. Если предлагается несколько вариантов, нужно выбрать один (или вставить свой).

fāmt=at=o?			
fām	-t	=at	=o
understand	? PST / PRS.3SG / INF / 2SG / COORD2	? COORD2 / 2SG / PQP	? Q / DISJ / COORD3

Например, на скриншоте выше для суффикса *-t* предложено несколько вариантов разбора: PST, PRS.3SG, INF и так далее. Нужно выбрать верный вариант, удалив все остальные варианты и знак вопроса.

		00:00:33.000				00:00:33.000				00:00:34.000					
A_sentence-txt-sgh [1]															
A_sentence-txt-ru [1]															
A_word-txt-sgh [10]		ðůvd,		fāmt=at=o?						Bād					
A_morph-txt-sgh [18]		ðův		-d		fām		-t		=at		=o		bād	
A_morph-gls-en [18]		gather		PRS.3SG		underst		PST		2SG		Q		then	
A_morph-pos-en [0]															
A_orig-txt-sgh															

Шаг 6. Внимательно проверьте содержимое слоя с глоссами:

- ✓ для всех морфем проставлены глоссы и леммы (для частотных — согласно [документу с глоссами](#), для прочих — по смыслу)
- ✓ ни в одной морфеме вы не забыли убрать знак вопроса ? или слэш /, которые ставит авто-токенизатор
- ✓ если где-то при глоссировании у вас возникли вопросы, которые вы хотите решить позже или вообще не можете решить (непонятная лексема, непонятная форма), то вместо глоссы в аннотации стоит знак вопроса ?
- ✓ случаи кодсвитчинга, где из другого языка взято одно или несколько слов (а не пара предложений), тоже отглоссированы по смыслу
 - например: *abor ši nonpobuem* отглоссировано как: *once now we_will_try*
- ✓ для неразборчивых фрагментов (записанных в первом слое как [?]) и для слов, на которых говорящая запнулась, глосса не проставлена (а если там была пустая аннотация, то она удалена)

Расстановка POS-тегов

Шаг 7. Сохраните eaf-файл и убедитесь, что он лежит в той же папке, где находится скрипт POSTAG.py. Запустите скрипт. (Чтобы запустить скрипт, войдите в командную строку в этой папке и введите `python POSTAG.py`.)

Скрипт попросит вас ввести название нужного файла. Введите название и нажмите Enter. Если скрипт сработал корректно, он напишет сообщение о том, сколько POS-тегов он предложил. После запуска скрипта в папке появится новый файл с названием `<прежнее_имя_файла>_POS_<время>.eaf`. Это ваш текст, но теперь ещё и с POS-тегами!

A_sentence-txt-sgh [1]						
A_sentence-txt-ru [1]						
A_word-txt-sgh [10]	ðũvd,		fāmt=at=o?			Bād
A_morph-txt-sgh [18]	ðũv	-d	fām	-t	=at	=o
A_morph-gls-en [18]	gather	PRS.3SG	underst	PST	2SG	Q
A_morph-pos-en [17]		V.AGR	V	T	CL.AGR	PRT
A_orig-txt-sgh [0]						

Шаг 8. Скрипт попытался сам добавить в текст POS-теги (в слой `A_morph-pos-en`). Если аннотация с тегом начинается с символа ?, значит, здесь анализатор не уверен. Скорее всего, в такой аннотации будет предложено несколько вариантов; самый вероятный будет первым. Если предложен только один вариант, то он, скорее всего, верен (если я что-то не напутал в словаре, конечно!). Там, где анализатор не уверен, нужно внимательно проверить. Если предлагается несколько вариантов, нужно выбрать один (или вставить свой).

A_sentence-txt-sgh [1]						
A_sentence-txt-ru [1]						
A_word-txt-sgh [10]	ðũvd,		fāmt=at=o?			Bād
A_morph-txt-sgh [18]	ðũv	-d	fām	-t	=at	=o
A_morph-gls-en [18]	gather	PRS.3SG	underst	PST	2SG	Q
A_morph-pos-en [18]	V	V.AGR	V	T	CL.AGR	PRT
A_orig-txt-sgh [0]						

Шаг 9. Внимательно проверьте содержимое слоя с POS-тегами:

- ✓ для всех морфем проставлены соответствующие POS-теги
- ✓ ни в одном теге вы не забыли убрать знак вопроса ? или слэш /, которые ставит авто-токенизатор
- ✓ если где-то при расстановке тегов у вас возникли вопросы, которые вы хотите решить позже или вообще не можете решить (непонятная лексема, непонятная форма), то вместо тега в аннотации стоит знак вопроса ?

- ✓ случаи кодсвитчинга получили соответствующий тег (RUS, TAJ, ENG)
- ✓ для неразборчивых фрагментов (записанных в первом слое как [?]) и для слов, на которых говорящая запнулась, POS-тег не проставлен (а если там была пустая аннотация, то она удалена)

Финализация

Шаг 10. Сохраните eaf-файл и убедитесь, что он лежит в той же папке, где находится скрипт CHECK.py. Запустите скрипт. (Чтобы запустить скрипт, войдите в командную строку в этой папке и введите `python CHECK.py`.)

Скрипт проверит ваш финальный файл на ошибки, которые могли появиться в процессе правки. Все ошибки отобразятся на экране:

```
D:\Учёба\Шугнанский\Шугнанский текст разбор\tokenization>python CHECK.py
Введите название файла .eaf: sgh_example_glossed_POS_2026-02-10T024403

> Есть предложения, в которых слой с текстом и слой с токенами не совпадают (1)!
> предложение: 'čīz kiĭt... kayŭn ōvd, fāmt=at=o? Bād mi=ti aray sabad yamand=atā,'
  токены:      'čīz kiĭt... kayŭn ōvd, fāmt=at=o? Bād mi=ti aray sabad yam-and=atā,'

> Есть токены, у которых морфемы не соответствуют тексту токена (2)!
> токен (без пунктуации): 'fāmt=at=o'      (контекст: '~kiĭt... kayŭn ōvd, fāmt=at=o? Bād mi=ti aray~')
  морфемы:      'fām-t=et=o'
> токен (без пунктуации): 'yam-and=atā'    (контекст: '~mi=ti aray sabad yam-and=atā,')
  морфемы:      'yam-and=atā'

> Есть морфемы, где вы забыли убрать знак вопроса (1)!
> морфема: 'čīz'      (контекст: 'čīz kiĭt... kayŭn ōvd,~')
  глосса:  'thing'
  POS-тег: '? PRO / N'

> Есть морфемы с пустыми глоссами (1)!
> морфема: 'nist'      (контекст: 'Kilometr=and nist!')
  глосса:  ''
  POS-тег: ''

> Есть морфемы с пустыми POS-тегами (1)!
> морфема: 'kilometr'  (контекст: 'Kilometr=and nist!')
  глосса:  'kilometer'
  POS-тег: ''
```

Постарайтесь исправить ошибки и запустить скрипт ещё раз. В конце должно появиться такое сообщение:

```
D:\Учёба\Шугнанский\Шугнанский текст разбор\tokenization>python CHECK.py
Введите название файла .eaf: sgh_example_glossed_POS_2026-02-10T024403

Всё отлично! Ура!

:)

...Ещё один файл? Нажмите Enter.
Если хотите выйти, напечатайте что угодно. |
```